LABORATORIO 2

DISEÑO DE CÓDIGO – PROMEDIO DE ESTUDIANTES

MELISA GUZMÁN MUÑOZ

DISEÑO DE DATOS Y ALGORITMOS

EDWARD ALEXANDER FRANCIA GUZMAN

ESCUELA COLOMBIA DE INGENIERIA JULIO GARAVITO

BOGOTÁ, 2026

Descripción del problema:

Se necesita desarrollar un programa que calcule el promedio de n estudiantes, donde cada estudiante tiene m notas. El programa debe mostrar el promedio de cada estudiante, indicar si aprobó o no (con la regla promedio ≥ 3.0) y finalmente entregar una lista con los nombres de los estudiantes aprobados.

Adicionalmente, la lista de aprobados debe mostrarse ordenada alfabéticamente, utilizando el algoritmo Merge Sort.

Diseño:

El diseño del programa se basa en un ciclo que procesa estudiante por estudiante. Primero, se solicita al usuario la cantidad de estudiantes n y la cantidad de notas por estudiante m. Luego, se ejecuta un ciclo for que se repite n veces. En cada repetición se pide al usuario una línea con el nombre del estudiante y sus m notas, separadas por espacios, utilizando split (). El programa valida que la cantidad de datos ingresados sea exactamente m + 1 (nombre + m notas). Si la entrada no cumple, se solicita nuevamente hasta que sea correcta. Después, se acumulan las notas en una variable suma, se calcula el promedio dividiendo entre m y se compara con 3.0 para determinar si el estudiante aprueba. Si el estudiante aprueba, su nombre se guarda en la lista aprobados. Al final del programa, antes de mostrar la lista, se aplica el algoritmo Merge Sort para ordenar alfabéticamente los estudiantes aprobados y luego se imprime la lista final.

Requerimientos:

- **RF01:** El programa debe pedir la cantidad de estudiantes **n**.
- **RF02:** El programa debe pedir la cantidad de notas por estudiante **m**.
- **RF03:** El programa debe permitir ingresar, para cada estudiante, el nombre y sus **m notas** separados por espacios.
- **RF04:** El programa debe validar que se ingresen exactamente **m + 1 datos** por estudiante (nombre + m notas).
- **RF05:** El programa debe calcular el promedio de cada estudiante.
- **RF06:** El programa debe indicar si el estudiante aprueba o no con la regla: **promedio ≥ 3.0**.

- **RF07:** El programa debe guardar los nombres de los estudiantes aprobados y mostrarlos al final.
- **RF08:** El programa debe ordenar la lista de estudiantes aprobados alfabéticamente usando **Merge Sort**.

Historial de usuarios:

HU01: Ingresar la cantidad de estudiantes **n**.

HU02: Ingresar la cantidad de notas por estudiante **m**.

HU03: Leer los datos de cada estudiante en una línea: nombre + **m notas** separadas por espacios.

HU04: Verificar que se ingresaron exactamente **m + 1 datos**; si no, pedir los datos otra vez.

HU05: Sumar las **m notas** del estudiante.

HU06: Calcular el promedio: $promedio = \frac{suma}{m}$

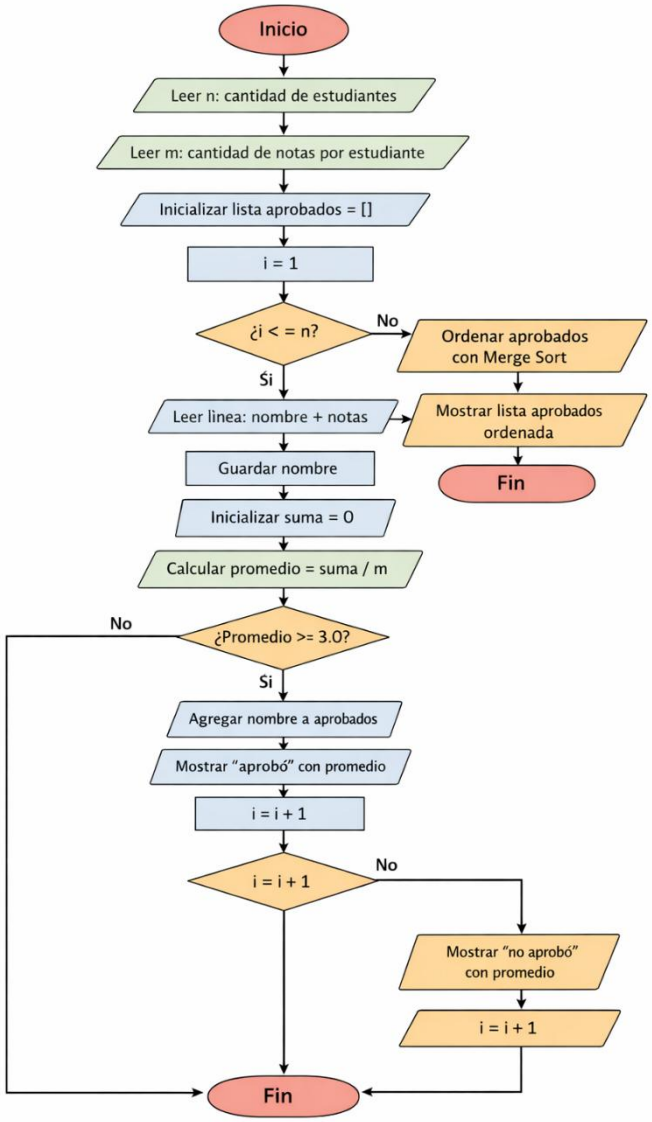
HU07: Si el promedio es ≥ 3.0, mostrar “aprobó” y guardar el nombre en aprobados.

HU08: Si el promedio es < 3.0, mostrar “no aprobó”.

HU09: Ordenar la lista de aprobados usando Merge Sort.

HU10: Mostrar la lista final de **"ESTUDIANTES APROBADOS ORDENADOS"**.

Diagrama de flujo:



Análisis de complejidad:

En la solución del problema se identifican principalmente estructuras iterativas como un ciclo for principal que se repite n veces (por cada estudiante). Dentro de este ciclo se encuentra un while que valida la entrada y un for interno que suma las m notas. Además, al final del programa se implementa el algoritmo Merge Sort para ordenar la lista de estudiantes aprobados, lo cual introduce una complejidad adicional de tipo logarítmica.

Tabla de Costo y Tiempo

def merge_sort (lista):

FUNCION	COSTO	TIEMPO
if len(*lista*) <= 1:	C1	1
mitad = len(*lista*) // 2	C2	2
izquierda = merge_sort(*lista*[: mitad])	C3	T(p/2)
derecha = merge_sort(*lista*[mitad:])	C4	T(p/2)
return merge (izquierda, derecha)	C5	M(p)

def merge (izq, der):

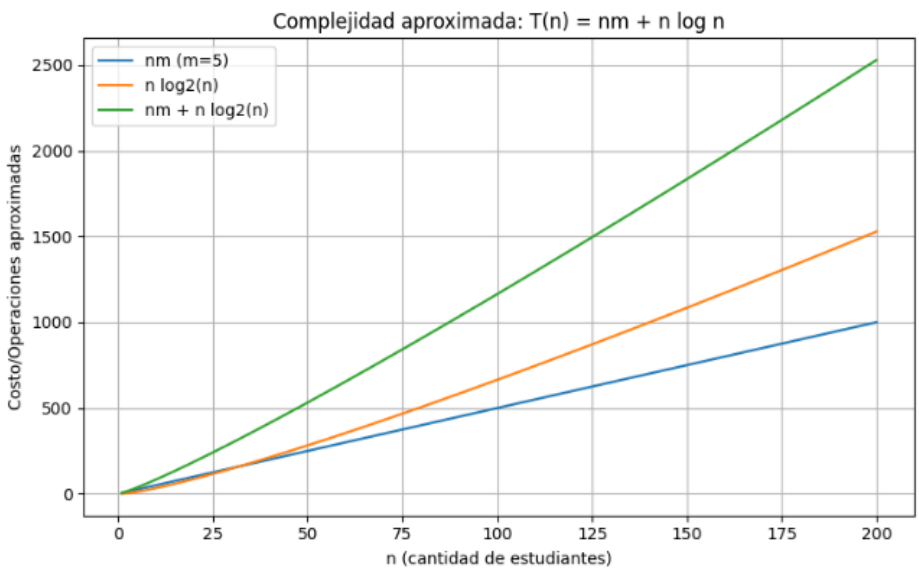
FUNCION	COSTO	TIEMPO
resultado = []	C6	1
i = 0	C7	1
j = 0	C8	1
while i < len(izq) and j < len(der):	C9	p
if izq[i] < der[j]:	C10	p
resultado.append(izq[i])	C11	p
i += 1	C12	p
resultado.append(der[j])	C13	p
j += 1	C14	p
resultado.extend(izq[i:])	C15	p
resultado.extend(der[j:])	C16	p
return resultado	C17	1

def est_prom():

FUNCION	COSTO	TIEMPO
n = int(input("Digite la cantidad de estudiantes: "))	C18	1
m = int(input("Digite la cantidad de notas que desea calcular por estudiante: "))	C19	1

aprobados = []	C20	1
print("Digite por cada estudiante: nombre y luego sus notas, \n" "si es decimal use punto (todo separado por espacios) y de enter entre cada estudiante: ")	C21	1
for i in range(n):	C22	n
datos = input("Ingrese los datos: ").split()	C23	n
while len(datos) != m + 1:	C24	$\sum_{i=1}^{n} t_i$
datos = input("Hubo un error, digite nuevamente nombre y notas separado por espacio: ").split()	C25	$\sum_{i=1}^{n} (t_i - 1)$
nombre = datos[0]	C26	n
suma = 0.0	C27	n
for j in range(m):	C28	n*m
suma += float(datos[j + 1])	C29	n*m
promedio = suma / m	C30	n
if promedio >= 3.0:	C31	n
aprobados.append(nombre)	C32	p
print(nombre, "Promedio:", promedio, "aprobó ☺")	C33	p
print(nombre, "Promedio:", promedio, "no aprobó :c")	C34	n-p
aprobados = merge_sort(aprobados)	C35	p * log p
print("ESTUDIANTES APROBADOS ORDENADOS:", aprobados)	C36	1
est_prom()	C37	1

$$T(n) = c1 + c2 + c3 * T\left(\frac{p}{2}\right) + c4 * T\left(\frac{p}{2}\right) + c5 * M(p) + c6 + c7 + c8 + c9 * p + c10 * p + c11 * p + c12 * p + c13 * p + c14 * p + c15 * p + c16 * p + c17 + c18 + c19 + c20 + c21 + c22 * n + c23 * n + c24 * (\sum_{i=1}^{n} t_i) + c25 * (\sum_{i=1}^{n} (t_i - 1)) + c26 * n + c27 * n + c28 * (nm) + c29 * (nm) + c30 * n + c31 * n + c32 * p + c33 * p + c34 * (n - p) + c35 * (plogp) + c36 + c37$$



Mejor caso:

$$t_i = 1$$

$$O(plogp)$$

$$n \cdot m$$

$$T_{mejor}(n, m, p) = n \cdot m + plogp$$

$$T_{mejor}(n, m) = O(nm + nlogn)$$

Peor caso:

$$\sum_{i=1}^{n} t_i$$

$$n \cdot m$$

$$plogp$$

$$T_{peor}(n, m, p) = n \cdot m + i = \sum_{i=1}^{n} t_i + plogp$$

$$t_i = k$$

$$\sum_{t=1}^{n} t_i = nk$$

$$T_{peor}(n, m, p) = n \cdot m + nk + plogp$$

$$T_{peor}(n, m) = O(nm + nk + nlogn)$$